Project Vitriol – NEA Design

Design Overview and Objectives

This project consists of two user-facing applications which share a common data model: a WPF-based map editor used to create, modify, and persist tile-based maps as structured JSON files, and a WPF runtime application responsible for loading a selected map and rendering it within a scrolling viewport where the player can navigate and trigger events.

A central architectural objective was to maintain strict stability of the map data format while allowing the editor and runtime to function independently. To achieve this, shared Data Transfer Objects (DTOs) and serialisation logic were designed first, establishing a unified schema before implementing editor and runtime behaviours. The editor and runtime are therefore treated as separate operational layers that consume the same map model, rather than directly depending on one another.

An alternative approach would have been to combine editor and runtime functionality into a single monolithic application. However, this was rejected due to increased coupling risks, reduced maintainability, and difficulty isolating rendering or logic faults. The layered separation adopted here ensures that issues in one subsystem do not propagate across the entire project.

Core Functional Objectives

- **Map Creation and Loading** – Allow users to create and load maps from a structured project directory (Vitriol.Projects).
- **Structured JSON Schema** – Map files must follow a predictable, versioned JSON schema to ensure forward compatibility and structural clarity.
- **Initial Map Generation** – New maps are initialised with a blank tile grid (default 32×32 tiles), with user-defined dimensions applied at creation.
- **Tileset Referencing** – Maps must reference tilesets through relative file paths. Tilesets can be loaded and unloaded in memory without invalidating existing tile indices, preserving map integrity.
- **Layer-Based Editing** – The editor must support:
    - Tile painting
    - Collision painting
    - Event placement
    - Undo and redo functionality (Ctrl+Z / Ctrl+Y)
- **Controlled Saving Behaviour** – The sample map supports continuous autosave. Newly created maps require at least one manual save before autosave is enabled. Manual save remains available at all times.

- **Runtime Playback Compatibility** – The runtime must load any valid map file and render only the visible region of the map. Viewport-based rendering must be implemented in both the editor and runtime to ensure consistent behaviour and performance.

Non-Functional Objectives

- **Performance** – Viewport-only rendering must be implemented to prevent unnecessary full-map redraw operations. Rendering must scale relative to visible area rather than total map size, ensuring responsiveness on minimum target hardware.
- **Reliability** – Validation and defensive checks must be applied when loading map files and tilesets. Invalid states must be detected before runtime execution.
- **Maintainability** – There must be clear architectural separation between:
    - Data model (Shared DTOs)
    - Editor logic
    - Runtime logic

  This separation reduces coupling, supports scalability, and prevents cascading failures across subsystems.
- **Usability** – The interface must remain minimal and intuitive. Editing workflows should reflect common conventions used in established map editors, reducing the learning curve for new users.

Data Design: ERD Progression

The data model was designed in clearly defined layers:

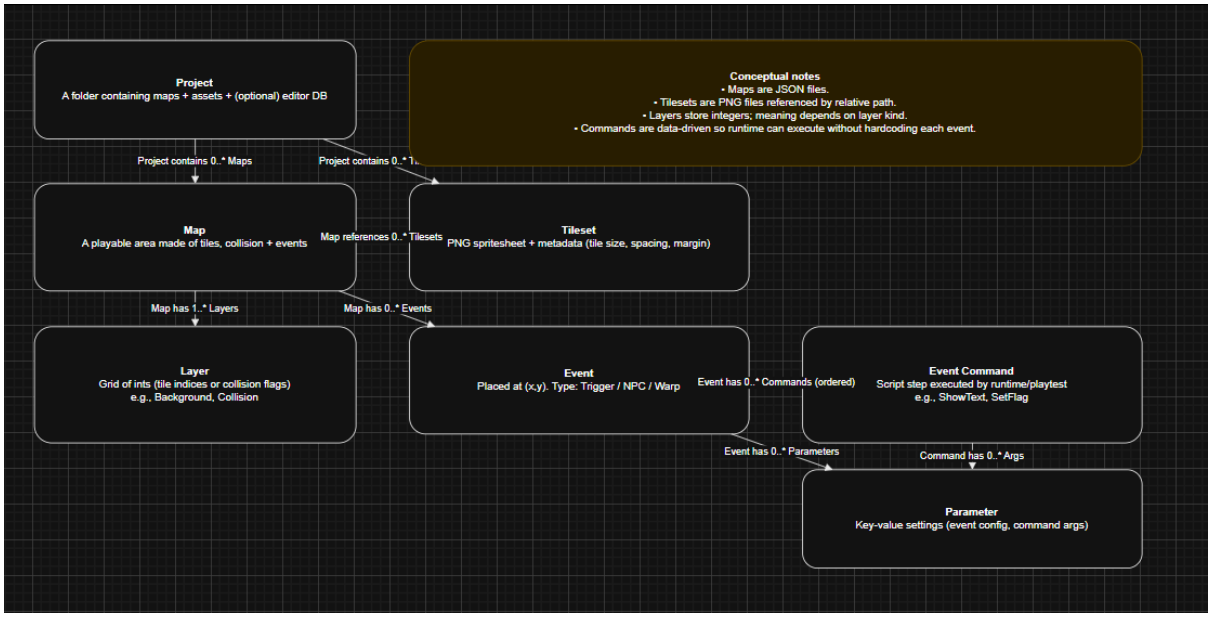
Conceptual -> Logical -> Physical (JSON) -> Persistence

This progression separates domain meaning from implementation detail. The conceptual model defines *what* exists in the system, the logical model defines *how* it is structured and constrained, the physical model defines *how it is encoded*, and the persistence view defines *how it is stored and accessed*.

Although map data is ultimately stored as JSON files, ERD-style modelling was still used during design to ensure structural clarity, scalability, and validation discipline. This layered modelling approach prevents implementation details from distorting domain reasoning.

Conceptual ERD: Domain Entities

Figure 01: Conceptual ERD – Core Entities and Relationships



At the conceptual level, the focus is purely on domain meaning. Storage format and technical encoding are intentionally excluded.

This level answers four essential domain questions:

- How large is a map and how is it identified?
- Which layers exist and what type of data does each store?
- Which tilesets provide the visual resources used by the map?
- Which events exist on the grid, and what behaviour occurs when they trigger?

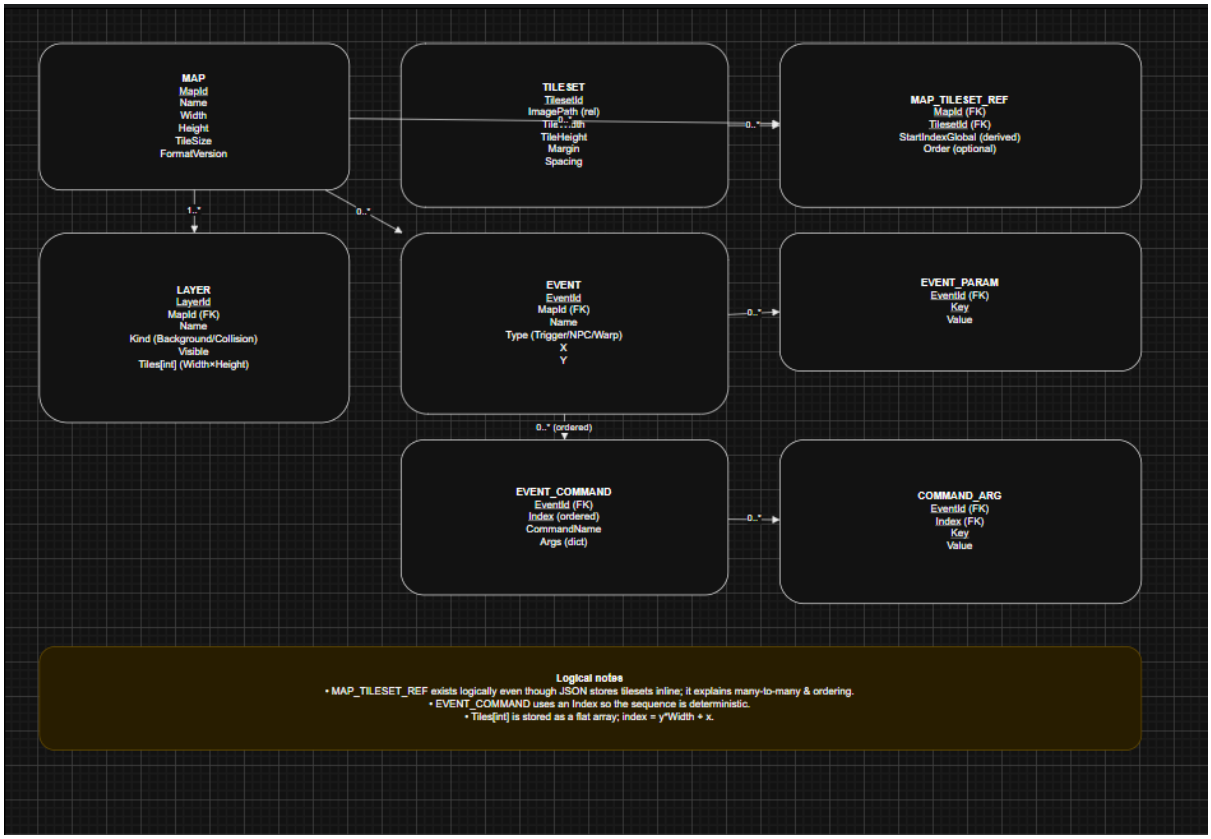
The primary conceptual entities are:

- **Map**
- **Layer**
- **Tileset**
- **Event**

By defining these entities independently of storage concerns, the design ensures that later implementation decisions (such as JSON structure) do not compromise conceptual clarity.

Logical ERD: Attributes and Keys

Figure 02: Logical ERD – Attributes, Keys, and Constraints



The logical stage introduces identifiers, attributes, and constraints aligned with runtime behaviour.

At this level:

- **Map** includes:
  - MapId (file-friendly identifier)
  - Name (display-friendly label)
  - Grid dimensions (Width, Height)
- **Layer** includes:
  - LayerId
  - Kind (e.g., background, collision)
  - Tile array sized Width × Height
- **TilesetRef** includes:
  - TilesetId
  - ImagePath
  - Layout metadata (TileSize, Margin, Spacing)

- **MapEvent** includes:
    - EventId (GUID)
    - Type (trigger, NPC, warp)
    - Position (X, Y)
    - Event-specific data

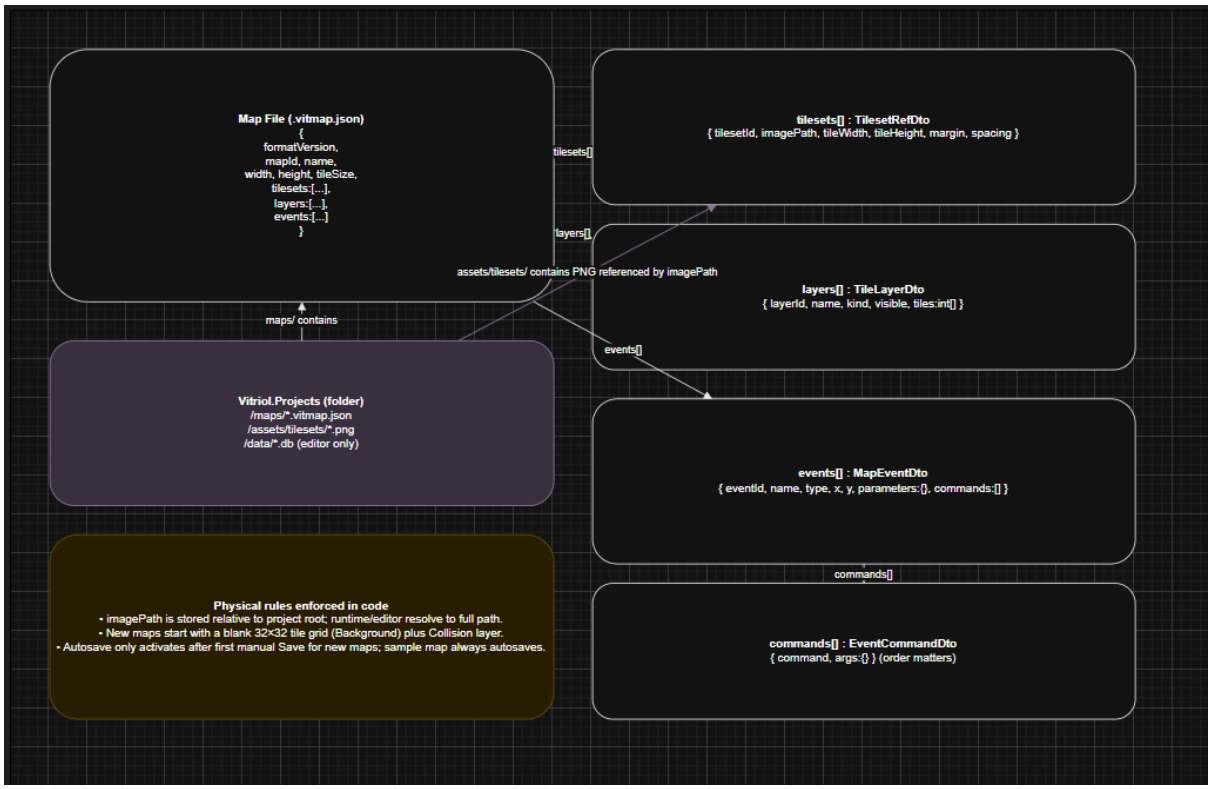
This logical definition introduces structural constraints that mirror runtime expectations. For example, tile arrays must align with declared map dimensions, and event positions must fall within map bounds.

Because the implementation uses C# and JSON, tile storage was designed using a **1D array rather than a 2D array**. This approach simplifies serialisation, reduces structural nesting in JSON, and enables faster index calculation using index = y * width + x. This decision improves both performance and structural clarity without affecting conceptual correctness.

An alternative 2D array representation was considered for readability; however, it was rejected due to increased serialisation complexity and reduced indexing efficiency.

Physical Design: JSON Schema Mapping

Figure 03: Physical Mapping – DTOs -> JSON File Structure



The physical model maps logical entities directly onto DTO records shared between the editor and runtime.

Each logical entity corresponds to a structured JSON object or array:

- Map metadata -> root JSON object
- Tile layer -> integer array
- Collision layer -> boolean or integer array
- Events -> list of event objects

Field names were kept consistent with DTO property names to minimise transformation logic and reduce deserialisation risk.

Over-normalisation was intentionally avoided. Since maps are loaded as complete documents rather than queried incrementally (as in relational systems), embedding related data within a single JSON file reduces dependency complexity and improves portability.

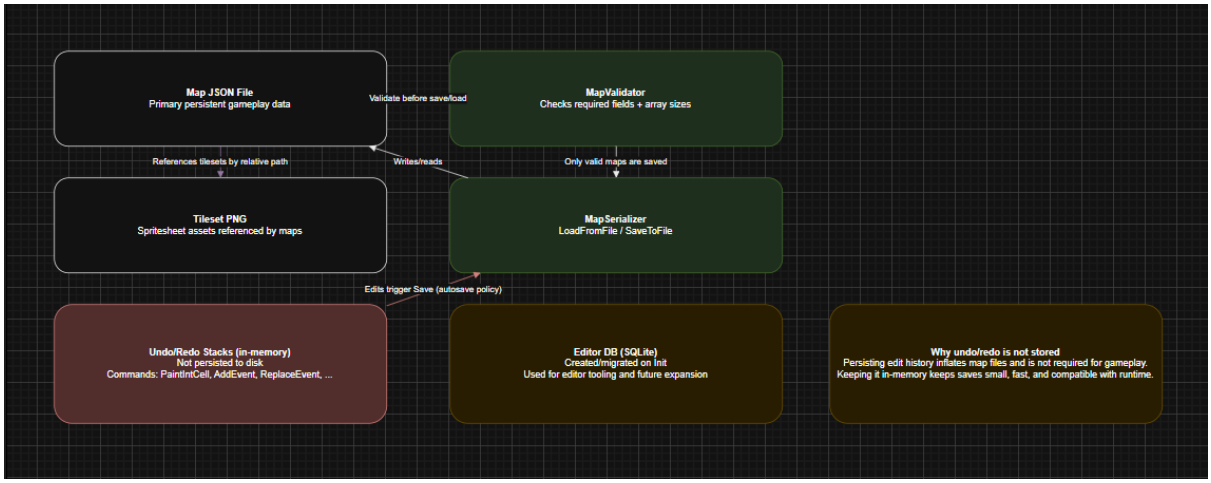
Key design decisions at this stage include:

- **FormatVersion Field** – Included to support future schema evolution without breaking compatibility.
- **Events Stored as a List** – Events are sparse relative to tile grids. Storing them as a list avoids allocating large, mostly empty structures.
- **Tileset References Stored per Map** – Each map maintains its own tileset reference to ensure portability between projects, provided required assets are available.

These decisions prioritise portability, scalability, and runtime-editor cohesion.

Persistence View: JSON Files + Project Metadata

Figure 04: Persistence ERD – File Storage and DB Responsibilities



The persistence layer distinguishes between map storage and project-level metadata.

Two forms of persistence exist within the system:

- **Map Persistence -** Maps are stored as JSON files under:
  Vitriol.Project/Maps/
  These files are written by the editor and read directly by the runtime.
- **Project Metadata Store -** The editor initialises lightweight project-level metadata to manage project configuration and structure. This is intentionally separate from individual map files.

This separation is deliberate.

Map files must remain:

- Easy to share
- Version-control friendly
- Self-contained
- Independent of editor internals

By isolating project-level concerns from map data, new editor-level features can be introduced without requiring structural changes to map files.

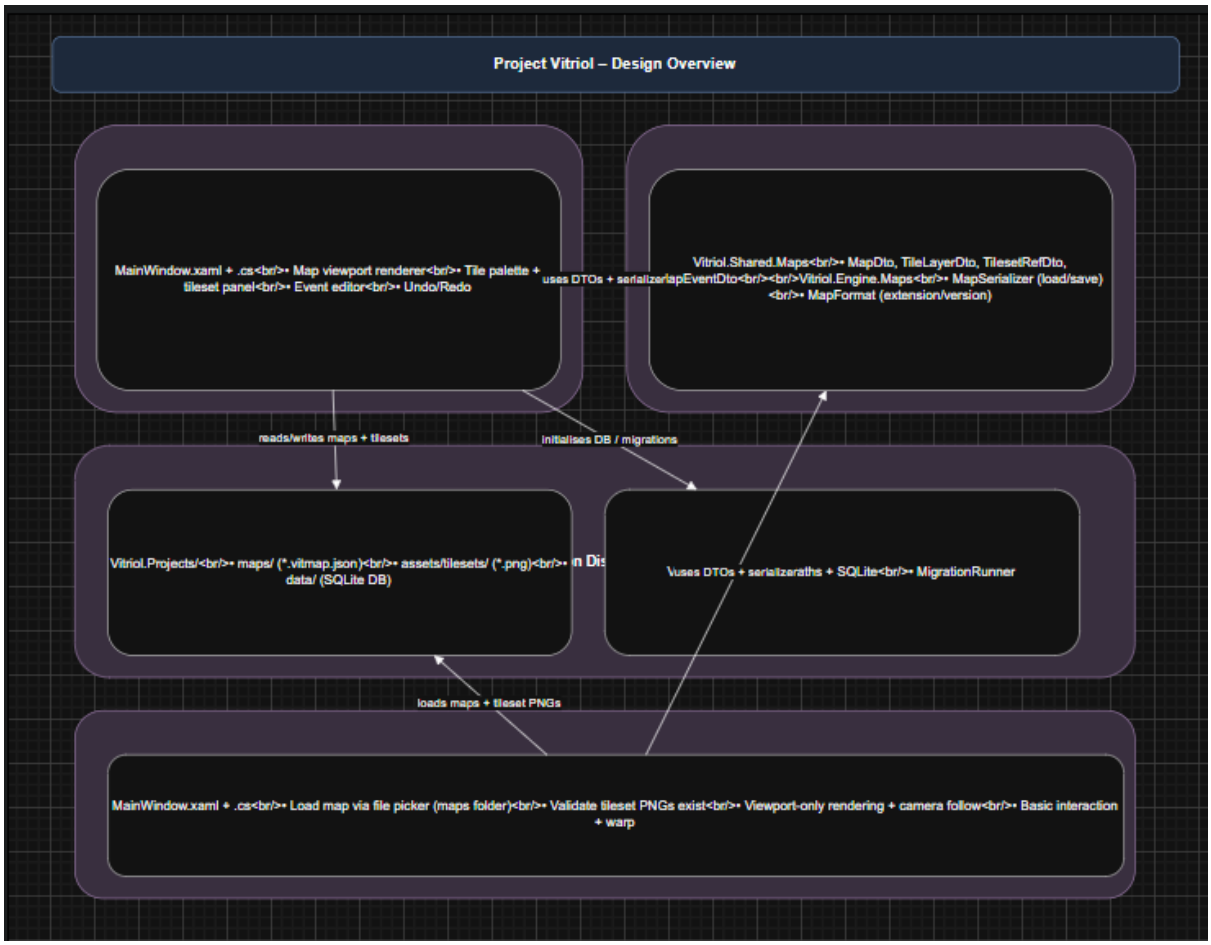
System Architecture: Project Structure and Layered Responsibilities

The system architecture is designed around strict separation of concerns. Shared map format definitions, DTOs, serialisation logic, and validation utilities are contained within shared assemblies. The editor and runtime applications depend on these shared components but maintain their own application-specific UI and behaviour layers.

This separation ensures consistency of data interpretation while preserving independence of execution environments.

High Level Architecture

Figure 05: Architecture Overview – Shared Libraries + Editor + Runtime



This diagram illustrates how both applications consume the same shared DTO definitions, serialisers, and validation logic.

Key architectural principles demonstrated here include:

- **Single Source of Truth** – There is one map schema definition.
- **Single Serialisation Pipeline** – Both applications use the same serialiser implementation.
- **Centralised Validation Logic** – Structural validation is not duplicated across applications.

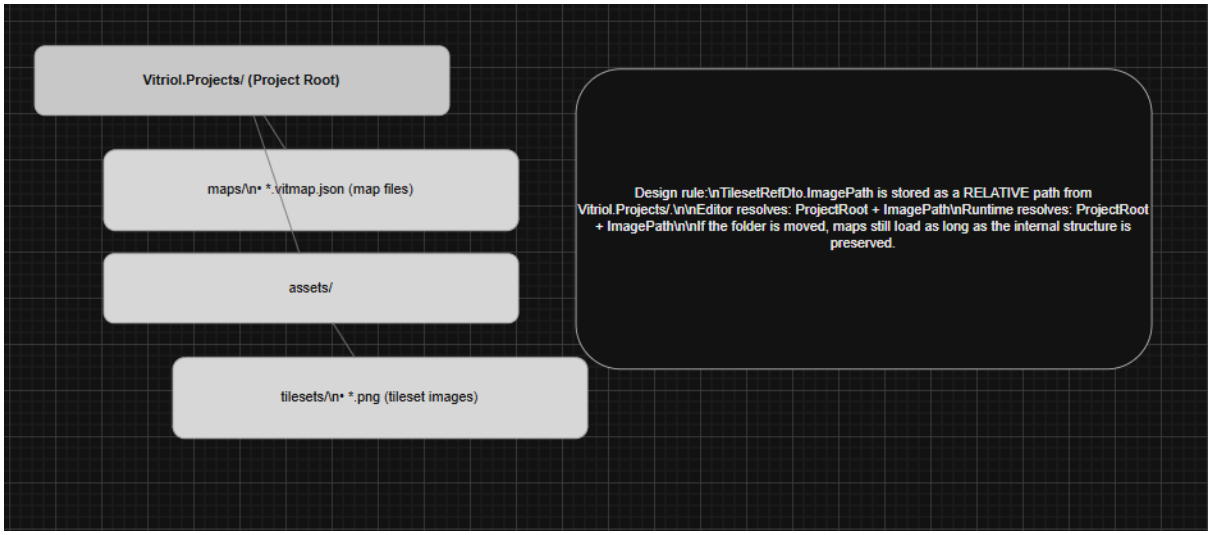
This eliminates divergence between editor and runtime behaviour. Consistency is enforced by shared architecture rather than informal development convention.

An alternative approach would have been to duplicate DTO definitions or allow the runtime to define its own interpretation of map data. This was rejected because it introduces schema drift risk and significantly increases maintenance complexity.

By enforcing dependency direction (Editor -> Shared, Runtime -> Shared), circular coupling is prevented. The runtime has no awareness of editor UI components, and the editor does not depend on runtime execution logic.

Project Folder Structure and Asset Conventions

Figure 06: Project Hierarchy – Vitriol.Projects Folder Layout



This diagram shows the standardised project directory layout:

- Vitriol.Projects/maps/ – Map JSON files
- Vitriol.Projects/assets/tilesets/ – Tileset PNG images

The folder structure is intentionally fixed to ensure both applications can deterministically compute file paths.

A key design rule is that tileset paths are stored as **relative paths from the project root**. This ensures:

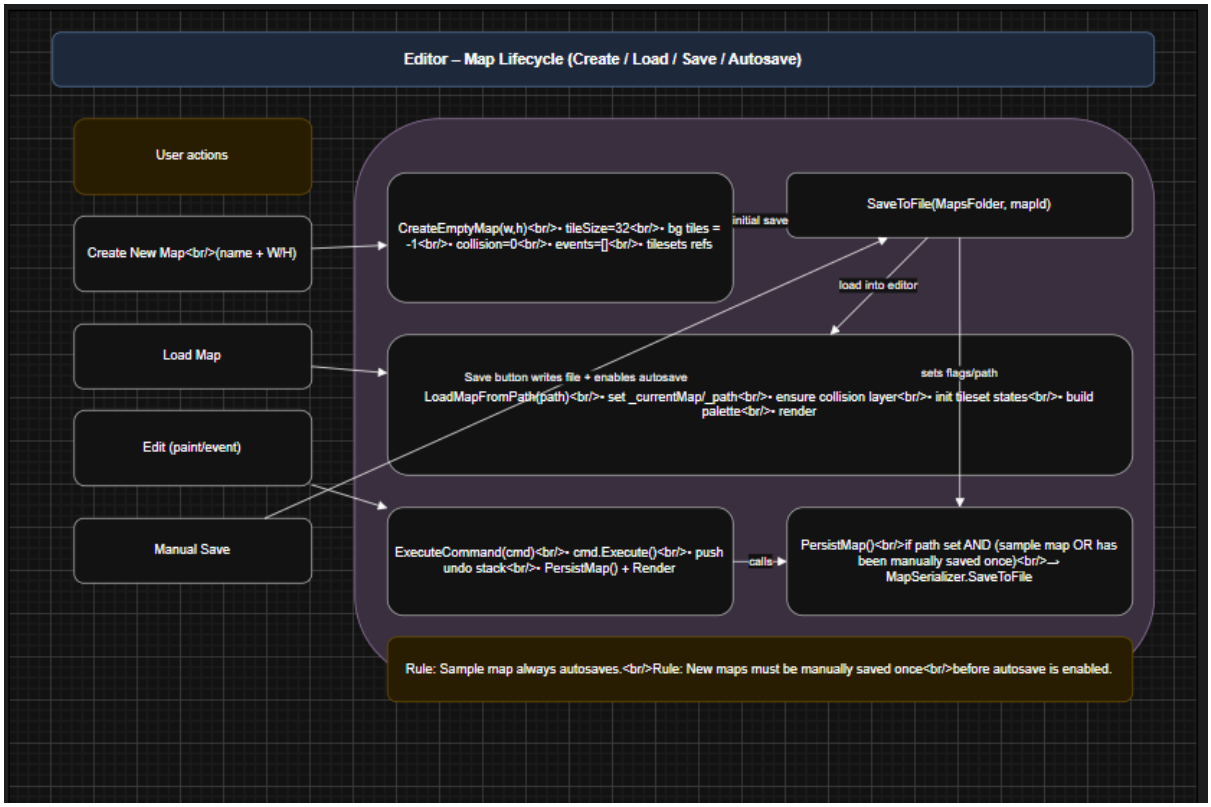
- Portability between machines
- Compatibility with version control systems
- Independence from absolute filesystem locations

As long as the internal folder structure remains consistent, maps remain valid when moved across devices.

Storing absolute paths was considered but rejected because it would tightly bind maps to a single machine's directory structure, significantly reducing portability and collaboration potential.

Editor Workflow Architecture

Figure 07: Editor Workflow – Create, Load, Save, Autosave Rules



This workflow diagram illustrates the lifecycle of a map within the editor:

- Creation
- Editing
- Manual save
- Autosave activation

The editor distinguishes between two operational contexts:

- **Sample Map** – Created for rapid experimentation and always autosaves. This supports immediate iteration during development and testing.
- **User-Created Maps** – Maps created via the “Create New Map” workflow must be manually saved at least once before autosave is enabled.

This design decision prevents:

- Accidental proliferation of unwanted files
- Unintended overwriting of unsaved work
- Loss of intentional filename control

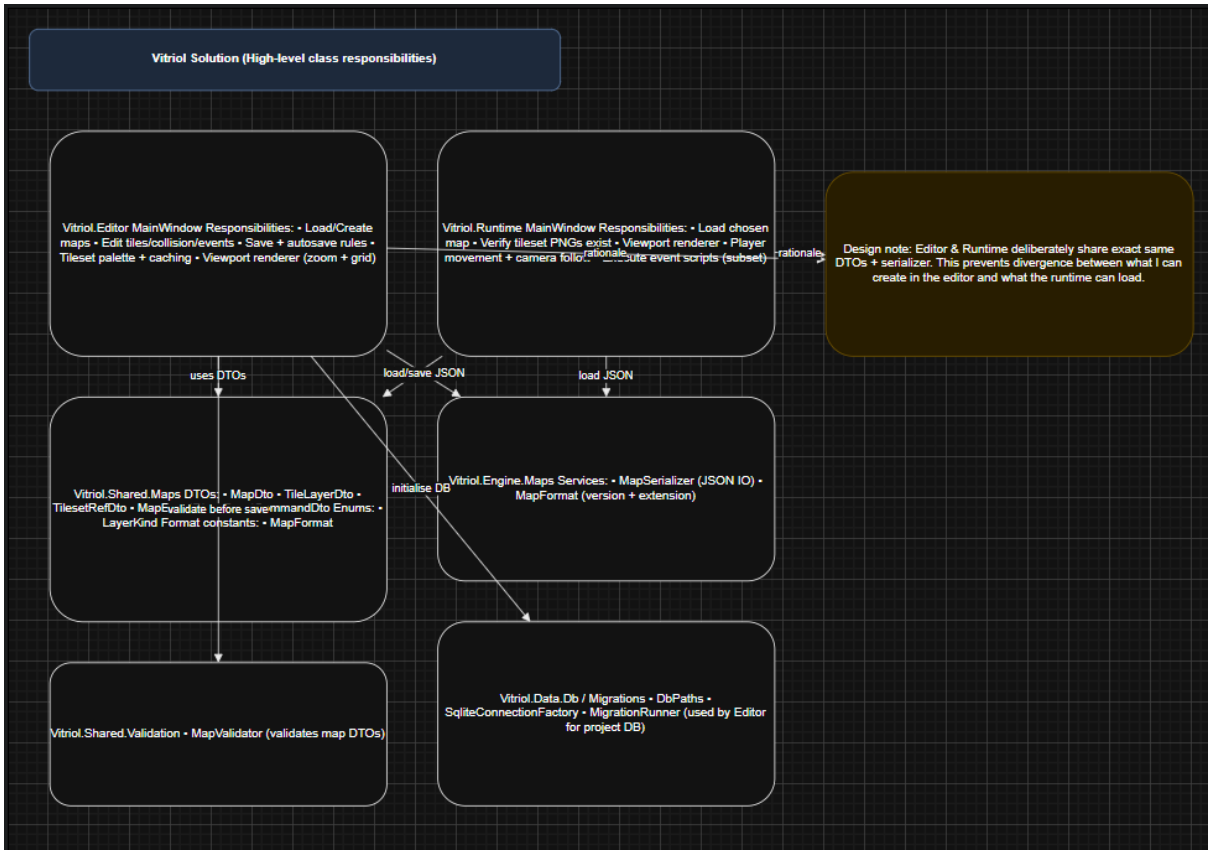
The requirement for an initial manual save ensures that each user map has a deliberate file identity before automated persistence begins.

This behaviour balances safety and efficiency: rapid testing remains frictionless via the sample map, while project structure remains controlled for user-defined maps.

Class Design: Engine, Editor, Runtime Separation

Class Overview: Shared DTOs and Services

Figure 08: Class Diagram – Shared Map Model, Serialisation, Validation



This diagram illustrates the shared map model, serialisation pipeline, and validation services consumed by both the editor and runtime applications.

A central design principle is that DTOs remain **passive data structures**. They contain no logic exclusive to either the editor or runtime. Their sole responsibility is representing structured data.

This constraint ensures:

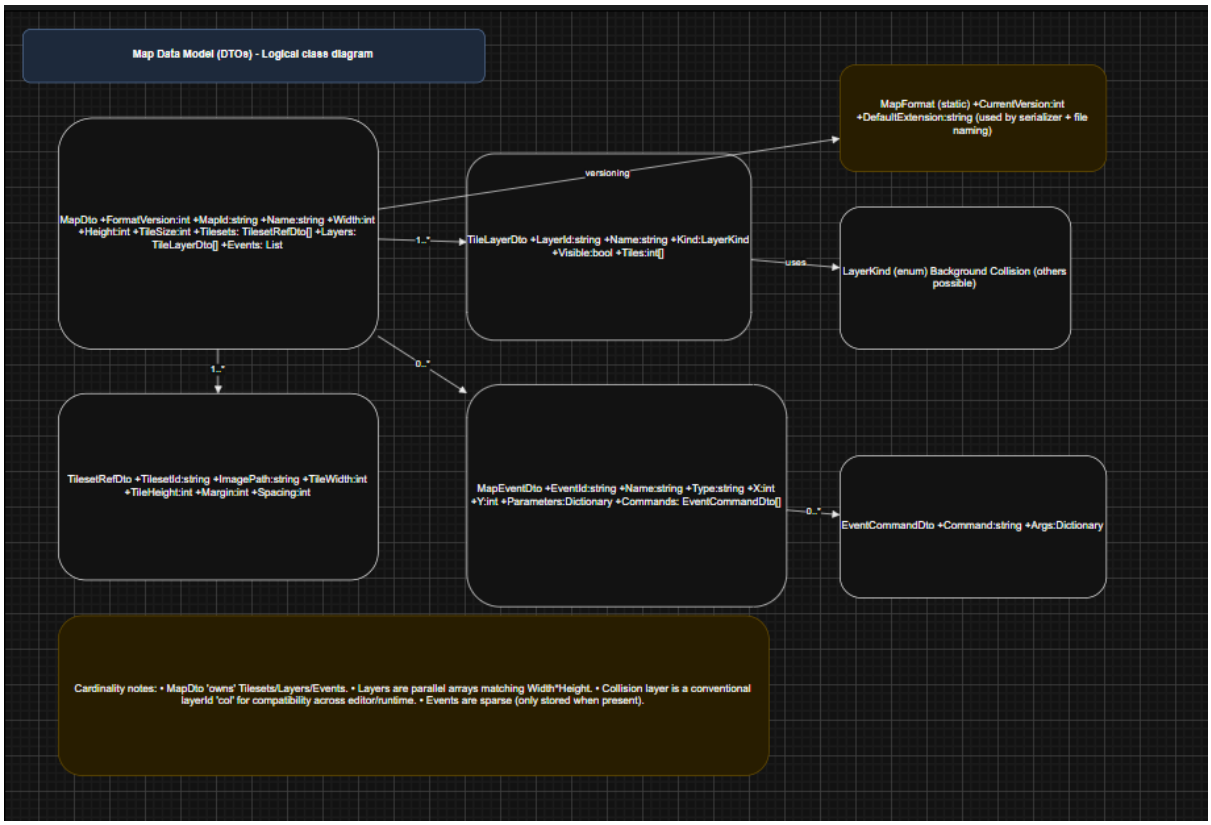
- Safe reuse across applications
- Predictable JSON serialisation behaviour
- Elimination of cross-layer logic leakage
- Reduced risk of circular dependencies

There is a single serialiser implementation and a single validation layer. Both applications consume these shared services rather than maintaining separate interpretations of the map format.

An alternative design would have embedded behaviour directly into DTO classes (for example, including rendering or validation logic within the model itself). This approach was rejected because it would tightly couple domain representation with application behaviour, reducing portability and increasing maintenance complexity.

Map Model and Layer + Event Composition

Figure 09: Class Diagram – MapDto, TileLayerDto, TilesetRefDto, MapEventDto



This diagram demonstrates how composition is prioritised within the map model.

The structure is intentionally hierarchical:

- MapDto contains:
    - Tile layers
    - Collision layer
    - Tileset reference
    - Event collection

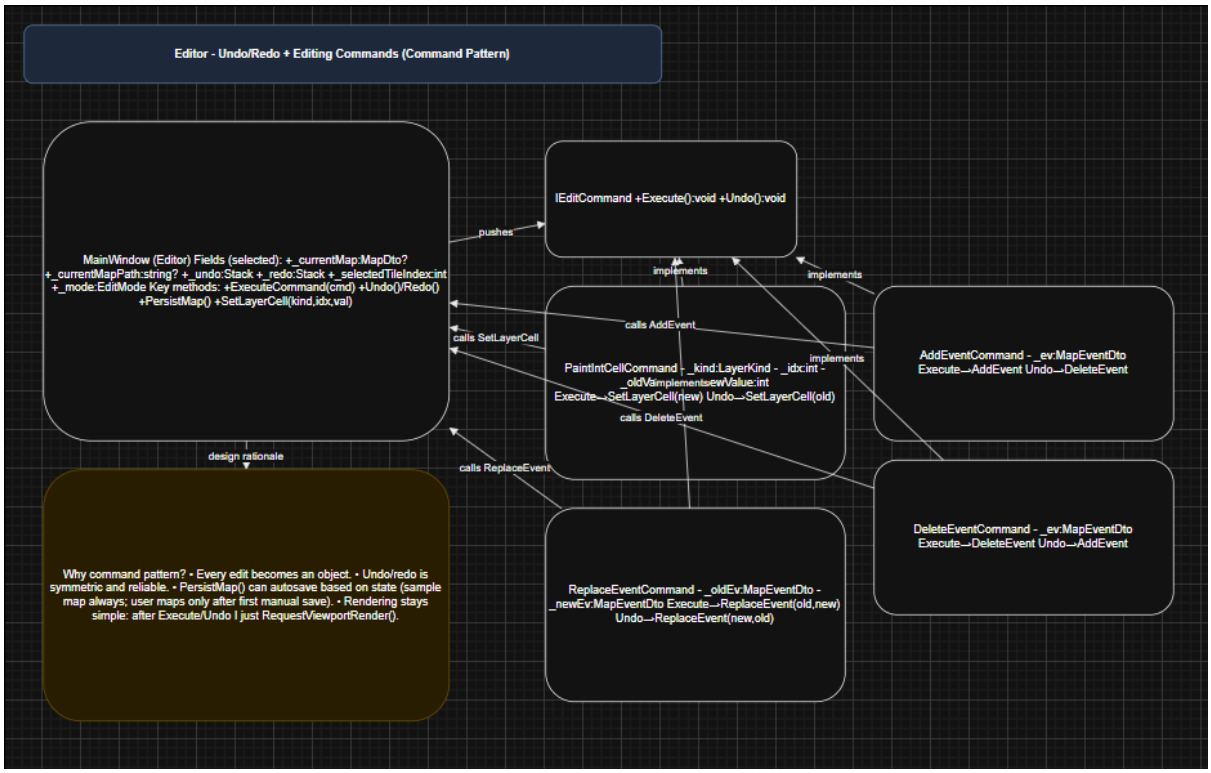
Design details include:

- **Layer Representation** – Layers are represented by TileLayerDto with a Kind property (e.g., visual or collision). This allows behaviour switching without introducing multiple subclass types, reducing class proliferation.
- **Event Extensibility** – Event scripts are represented as a list of EventCommandDto objects. Each command contains an argument dictionary, allowing new command types to be introduced without modifying the schema structure.
- **Null Normalisation on Load** – After deserialisation, the editor performs normalisation to ensure collections (events, commands, parameters) are never null. This simplifies UI logic by guaranteeing safe iteration.

Composition is preferred over inheritance throughout the model. This avoids unnecessary class hierarchies and keeps structural relationships explicit and predictable.

Editor Editing Model: Commands, Undo/Redo, Persistence

Figure 10: Class Diagram – Editor Commands and Undo/Redo Stack



This diagram shows the command-based editing system built around an IEditCommand interface, with concrete implementations such as:

- PaintIntCellCommand
- AddEventCommand
- DeleteEventCommand

- ReplaceEventCommand

Undo/Redo is implemented using the Command Pattern. This design was deliberately chosen because it:

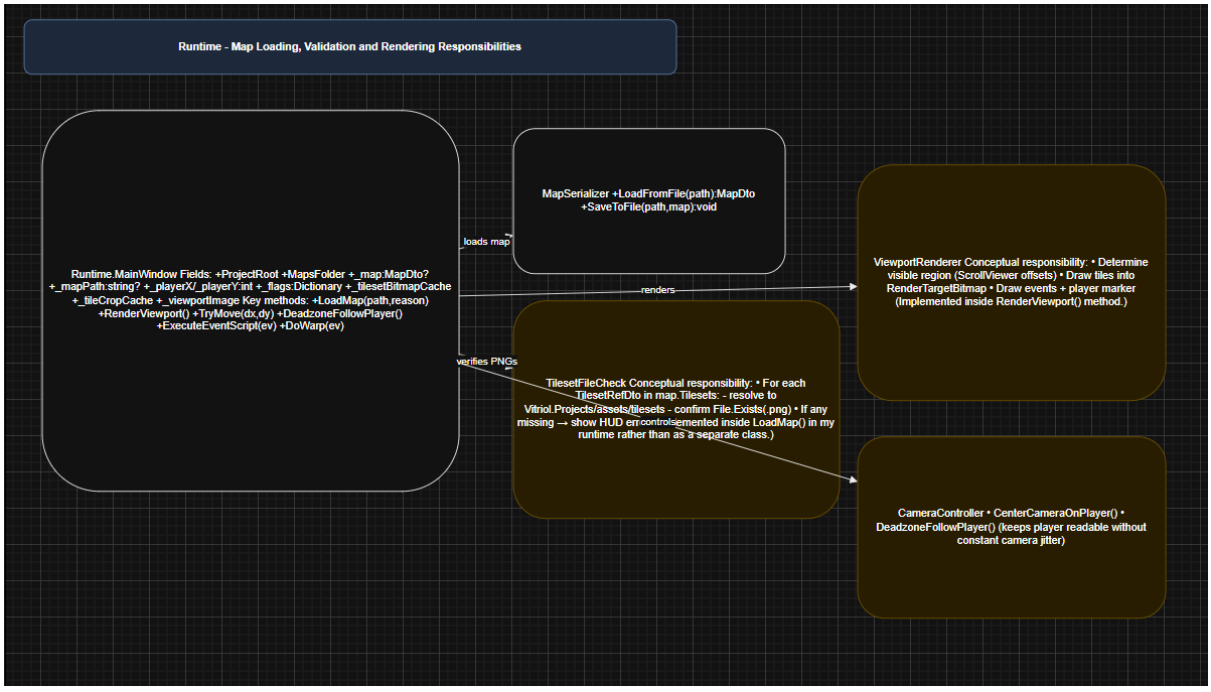
- Captures state changes in a reversible and explicit form
- Keeps UI code lightweight — the UI requests an action, and the command encapsulates execution logic
- Centralises modification behaviour, ensuring consistency across editing operations
- Simplifies autosave and persistence triggers by treating all edits uniformly

An alternative approach would have been to directly mutate the map model and manually track previous states. This was rejected because it complicates reversibility, increases coupling between UI and data logic, and risks inconsistent undo behaviour.

The command-based architecture ensures deterministic and maintainable editing state transitions.

Runtime Classes: Load, Render, Input, Camera

Figure 11: Class Diagram – Runtime Responsibilities (Loading, Rendering, Camera Follow)



This diagram illustrates separation within the runtime layer:

- Map loading logic
- Rendering subsystem
- Input handling
- Camera control

In the runtime architecture:

- **Map Loading** is isolated from rendering and input.
- **Rendering** consumes loaded data but does not modify it.
- **Input Handling** updates player state but does not directly manage rendering.
- **Camera Logic** calculates viewport bounds independently of input processing.

This separation prevents unnecessary asset reloads or cache rebuilding during gameplay. For example, player movement updates position state without triggering full asset reinitialisation.

An alternative tightly coupled runtime (where rendering and loading are intertwined) would increase the risk of performance instability and complicate debugging.

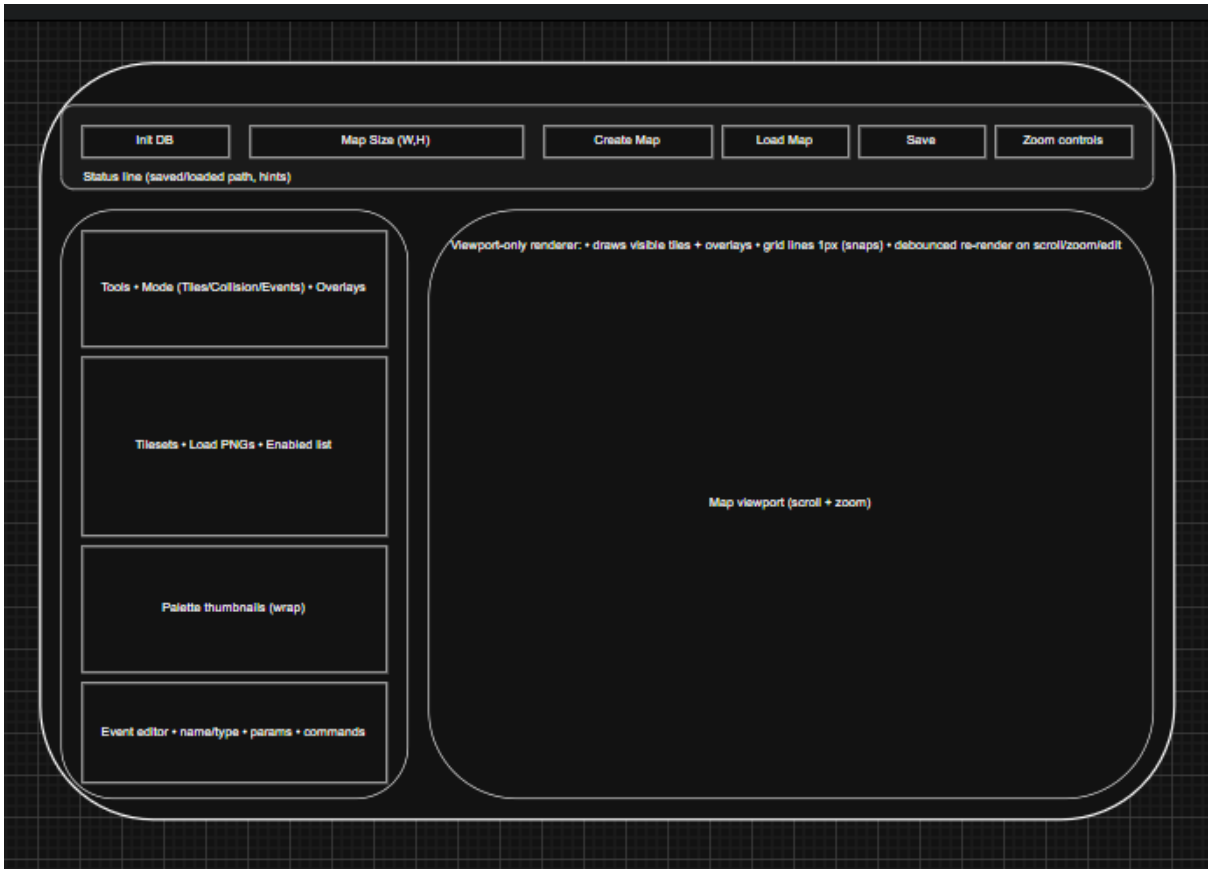
By separating these concerns, the runtime maintains:

- Deterministic update behaviour
- Efficient viewport-only rendering
- Clear control flow ownership

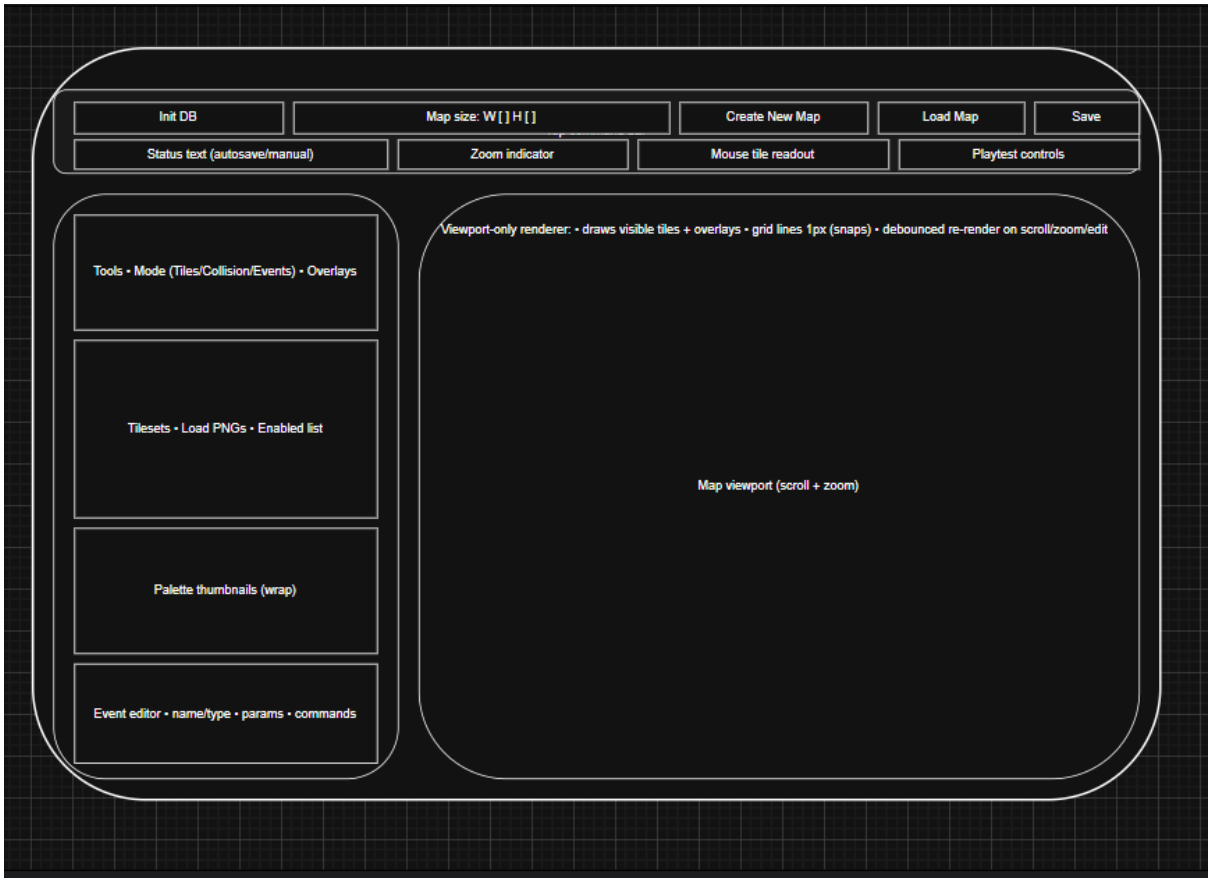
User Interface Design: Iterative UI Designs

Editor UI Iterations

Figures 12, 13, 14: Editor UI Iteration Set – A -> C


Init DB
Map Size (W,H)
Create Map
Load Map
Save
Zoom controls
Status line (saved/loaded path, hints)
Tools • Mode (Tiles/Collision/Events) • Overlays
Tilesets • Load PNGs • Enabled list
Palette thumbnails (wrap)
Event editor • name/type • params • commands
Viewport-only renderer: • draws visible tiles + overlays • grid lines 1px (snaps) • debounced re-render on scroll/zoom/edit
Map viewport (scroll + zoom)


Init Project DB
Map dimensions (W/H textboxes)
Create Sample
Create New
Load
Save
Undo/Redo
Zoom - / Reset / +
Playtest hint strip
Mouse tile info
Tools • Mode (Tiles/Collision/Events) • Overlays
Tilesets • Load PNGs • Enabled list
Palette thumbnails (wrap)
Event editor • name/type • params • commands
Viewport-only renderer: • draws visible tiles + overlays • grid lines 1px (snaps) • debounced re-render on scroll/zoom/edit
Map viewport (scroll + zoom)



These iterations demonstrate progressive refinement of the editor layout. Each version adjusted spatial hierarchy and control grouping to prioritise map editing efficiency.

The evolution moved toward a stable four-region layout:

- Top toolbar
- Left tools sidebar
- Central map viewport
- Expandable lower event editor panel

Main UI Decisions and Rationale

- **Viewport Priority** – A wide central viewport was prioritised to maximise spatial awareness during map editing. Map creation is inherently spatial; therefore, screen real estate was allocated primarily to the grid display rather than auxiliary controls.
- **Left-Aligned Tool Sidebar** – Editing tools were placed on the left to align with conventions used in established creative software (e.g., image editors and tile-based level editors). This reduces cognitive friction for users already familiar with such interfaces.
- **Top Toolbar Grouping** – The top toolbar groups actions into:
    - Project-level actions (create, load)

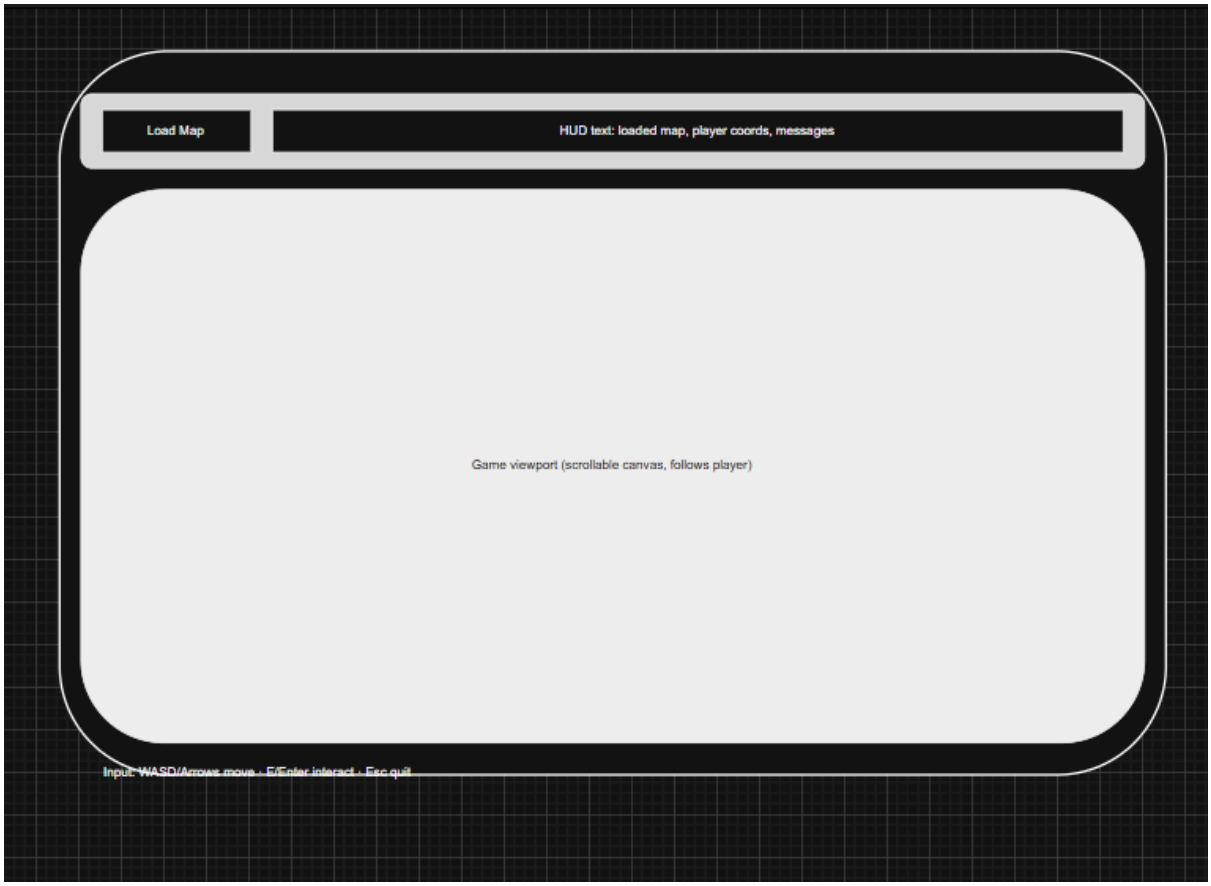
    - Map-level actions (save)
    - View-level actions (zoom controls)

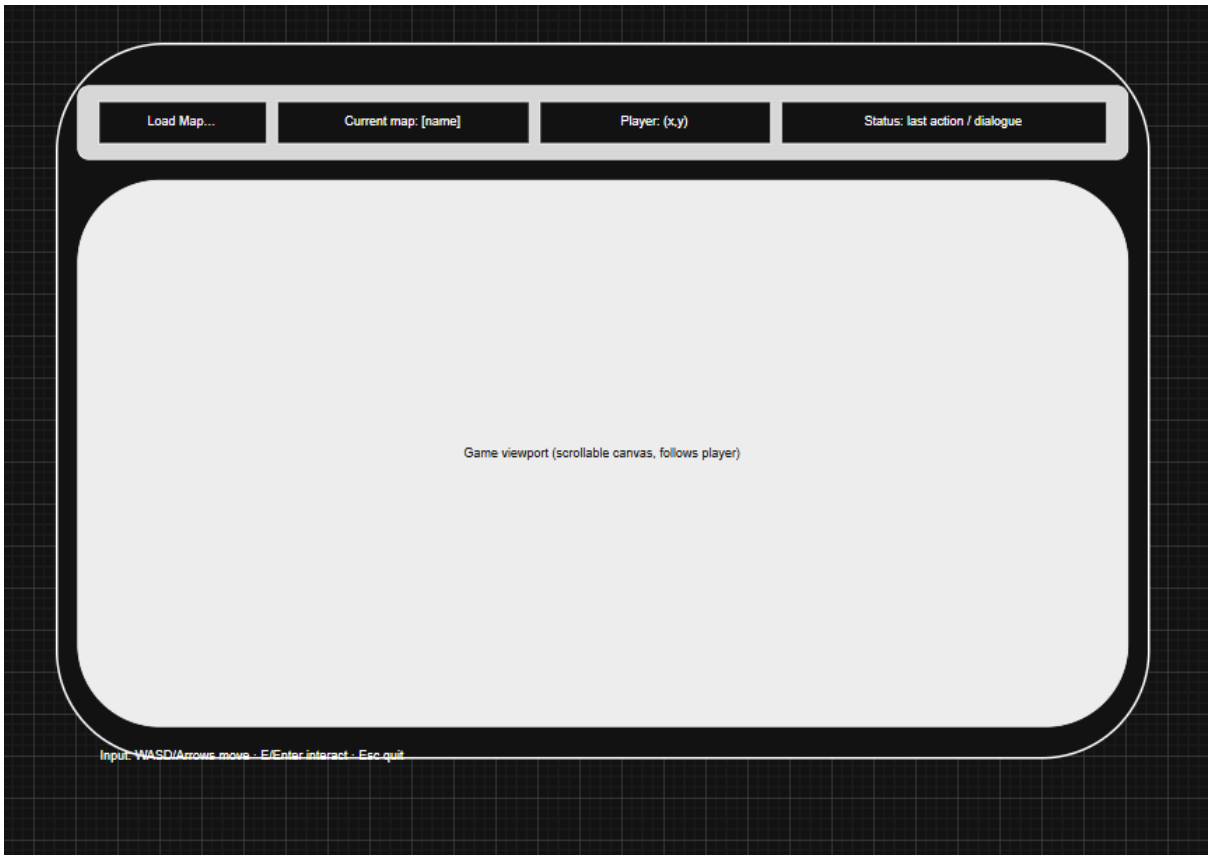
  This grouping reflects task hierarchy rather than technical implementation, improving discoverability.
- **Event Editor Placement** – The event editor is positioned below the tile palette because event configuration is a secondary workflow compared to tile painting. Tile editing occurs continuously during map construction, whereas event editing is typically intermittent.

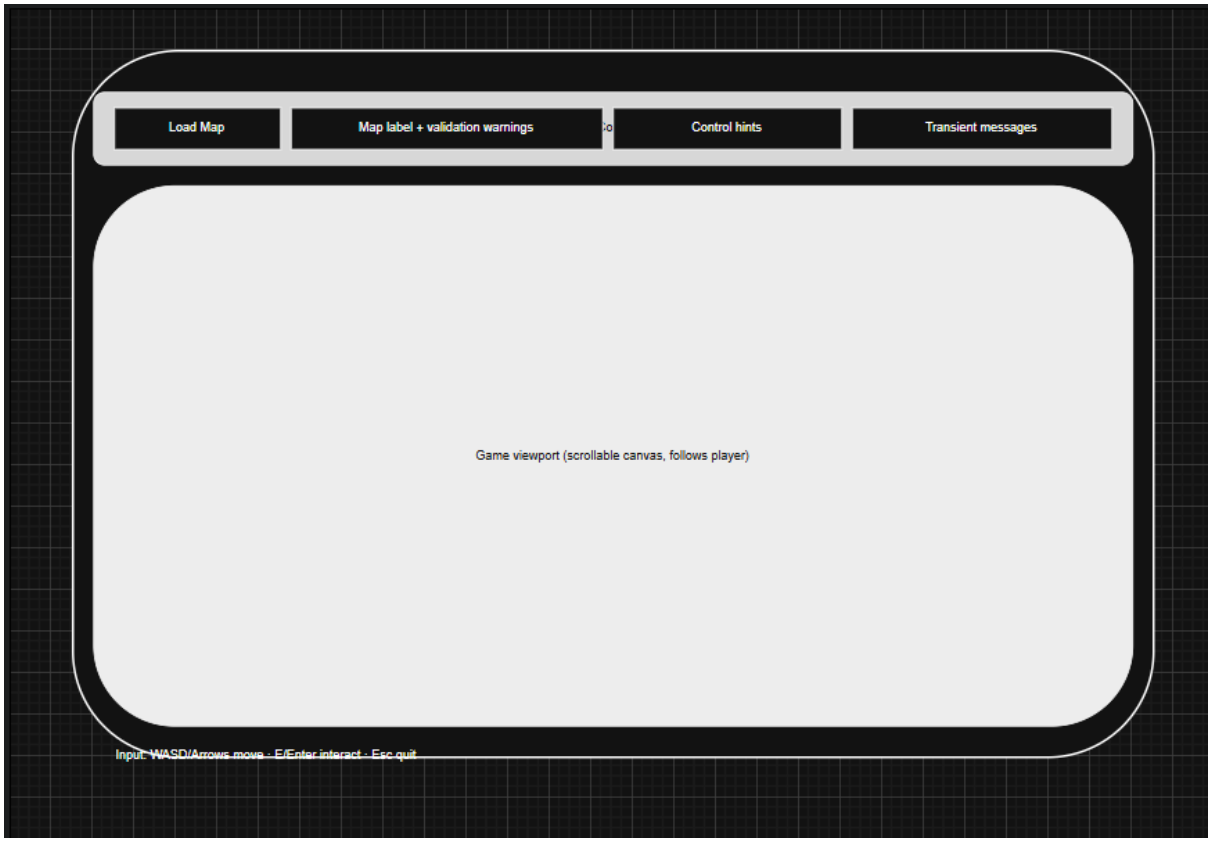
Earlier iterations experimented with alternative placements (including top docking and modal panels). These were rejected because they either reduced viewport area or disrupted editing flow.

Runtime UI Iterations

Figures 15, 16, 17: Runtime UI Iteration Set – A -> C


Load Map
HUD text: loaded map, player coords, messages
Game viewport (scrollable canvas, follows player)
Input: WASD/Arrows move · E/Enter interact · Esc quit


Load Map...
Current map: [name]
Player: (x,y)
Status: last action / dialogue
Game viewport (scrollable canvas, follows player)
Input: WASD/Arrows move · E/Enter interact · Esc quit



These iterations show the transition from an automatically loaded runtime model to an explicit, user-driven loading workflow.

Initially, the runtime loaded the most recently edited map automatically. While efficient for development testing, this approach limited user control.

The final design introduces:

- A **Load Map** button
- A minimal HUD displaying:
    - Current map
    - Player status
    - Control hints
- A central scrolling viewport

Runtime UI Design Principles

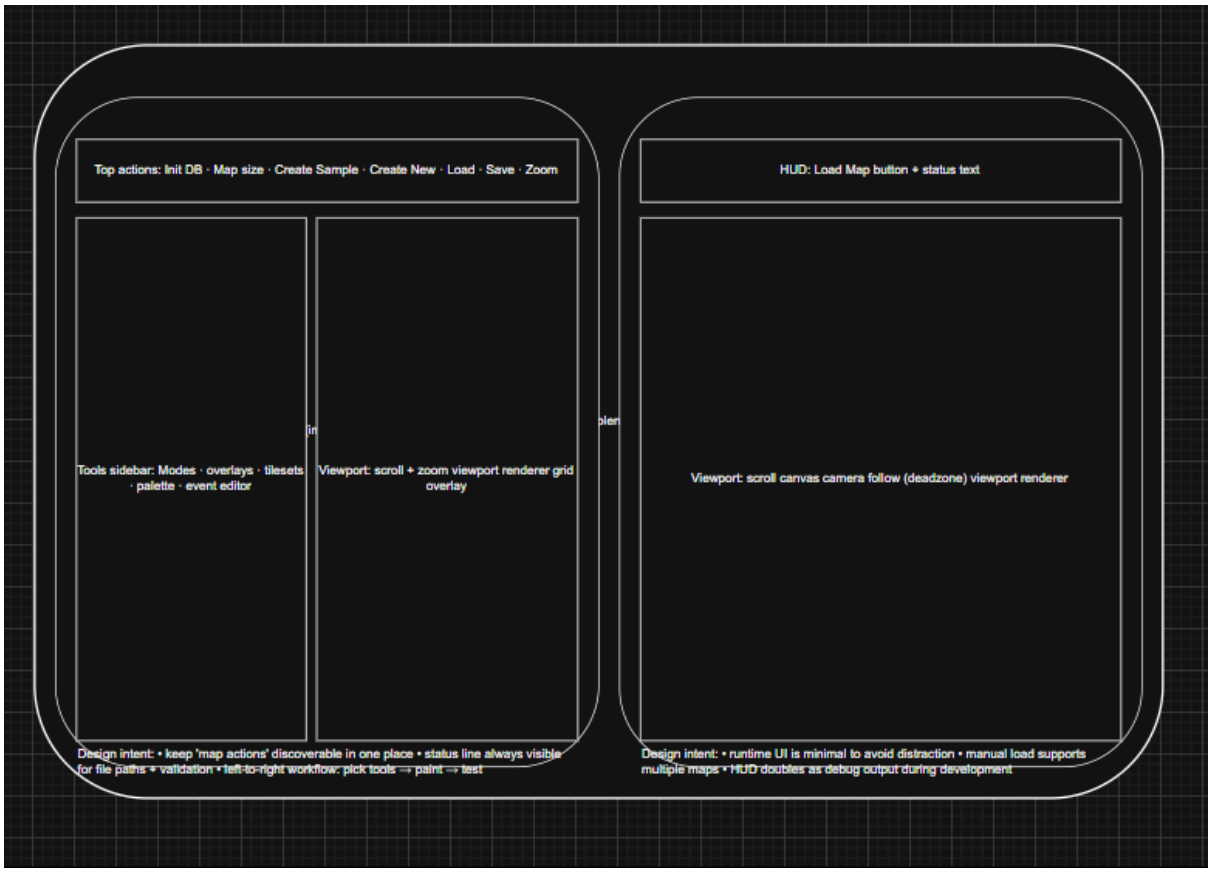
- **Intentional Minimalism** – The runtime interface avoids editor-style controls. It is focused exclusively on gameplay presentation.
- **Explicit User Choice** – The addition of the Load Map button allows the user to select which map to play at runtime. This increases flexibility and better reflects real-world deployment behaviour.

- **Separation from Editor Context** – The runtime does not assume any prior editing session state. It behaves independently of the editor application.

The progression from automatic loading to explicit loading demonstrates responsiveness to usability concerns and improved separation between development and runtime contexts.

Current UI Mapping

Figure 18: UI Control Mapping – XAML Controls -> Event Handlers



This diagram demonstrates traceability between design and implementation by mapping visual XAML controls to their corresponding event handlers and logic layers.

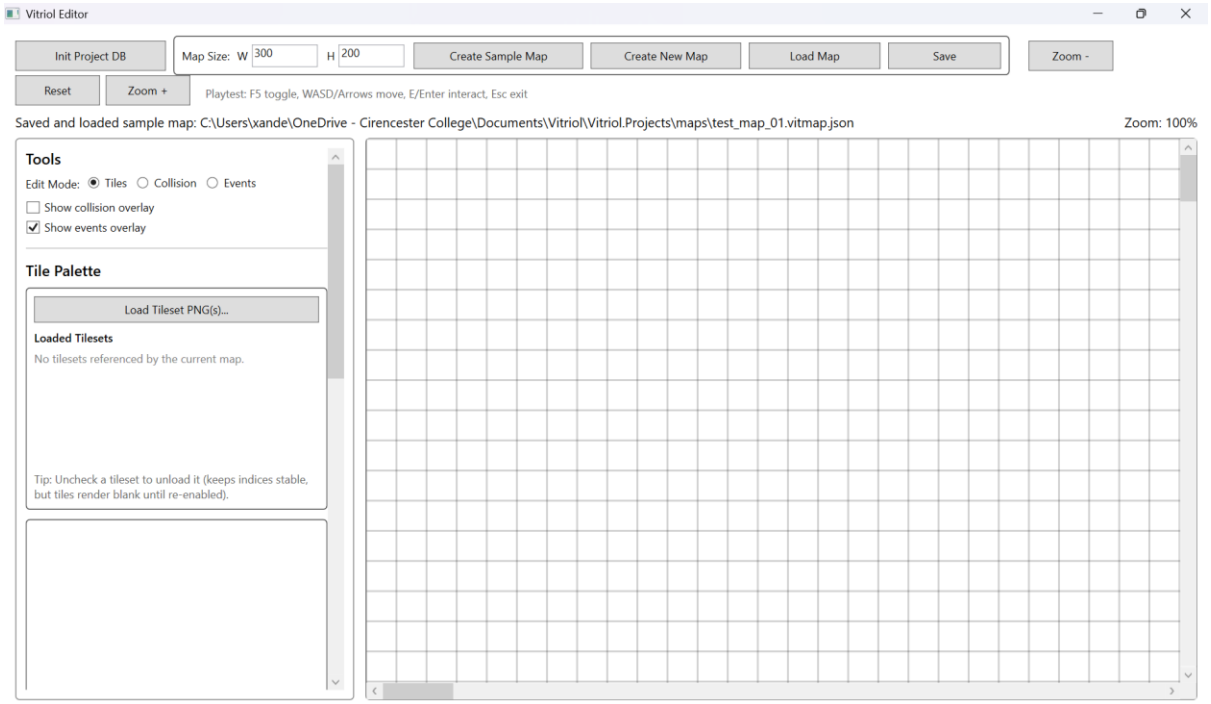
This ensures:

- Clear separation between presentation (XAML) and behaviour (code-behind or command handlers)
- Maintainable event routing
- Architectural transparency between UI and application logic

This mapping reinforces adherence to separation of concerns within the UI layer.

Current UI Screenshots

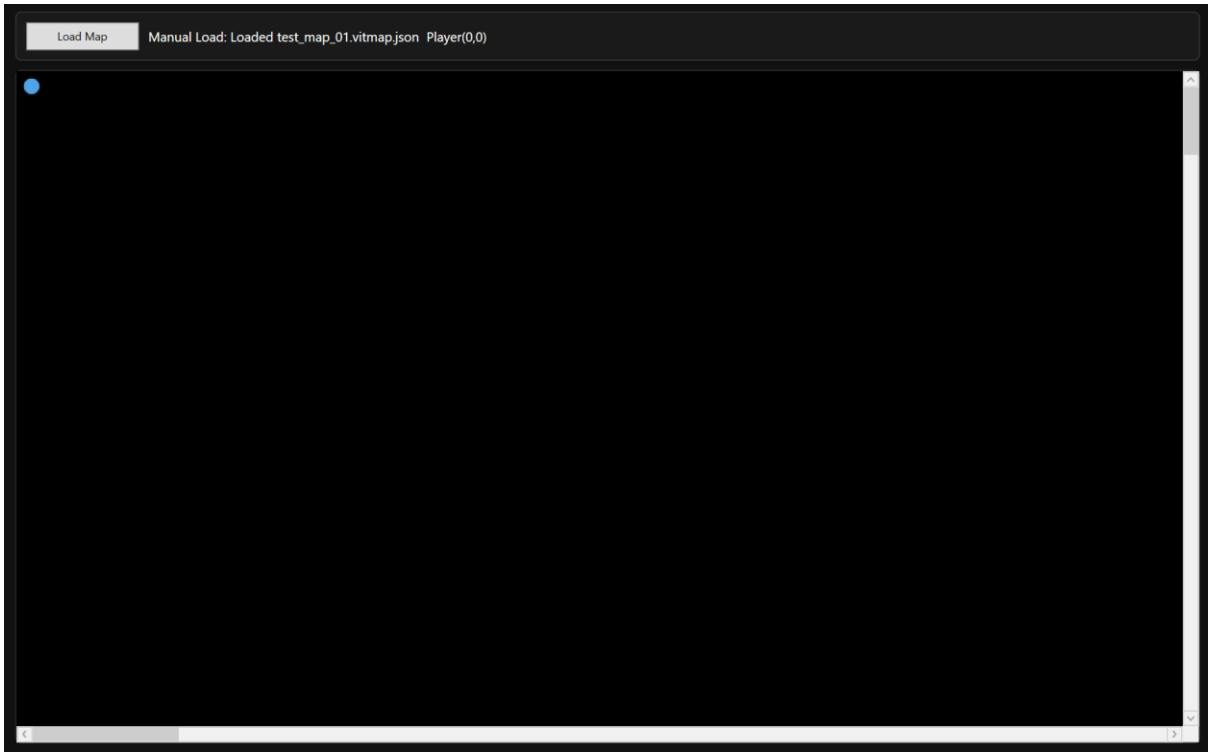
Figure X1: Current UI – WPF Editor



The final editor interface reflects the stabilised layout derived from iterative refinement:

- Wide viewport
- Persistent tools sidebar
- Structured top toolbar
- Status indicators

Figure X2: Current UI – WPF Runtime



The final runtime interface demonstrates:

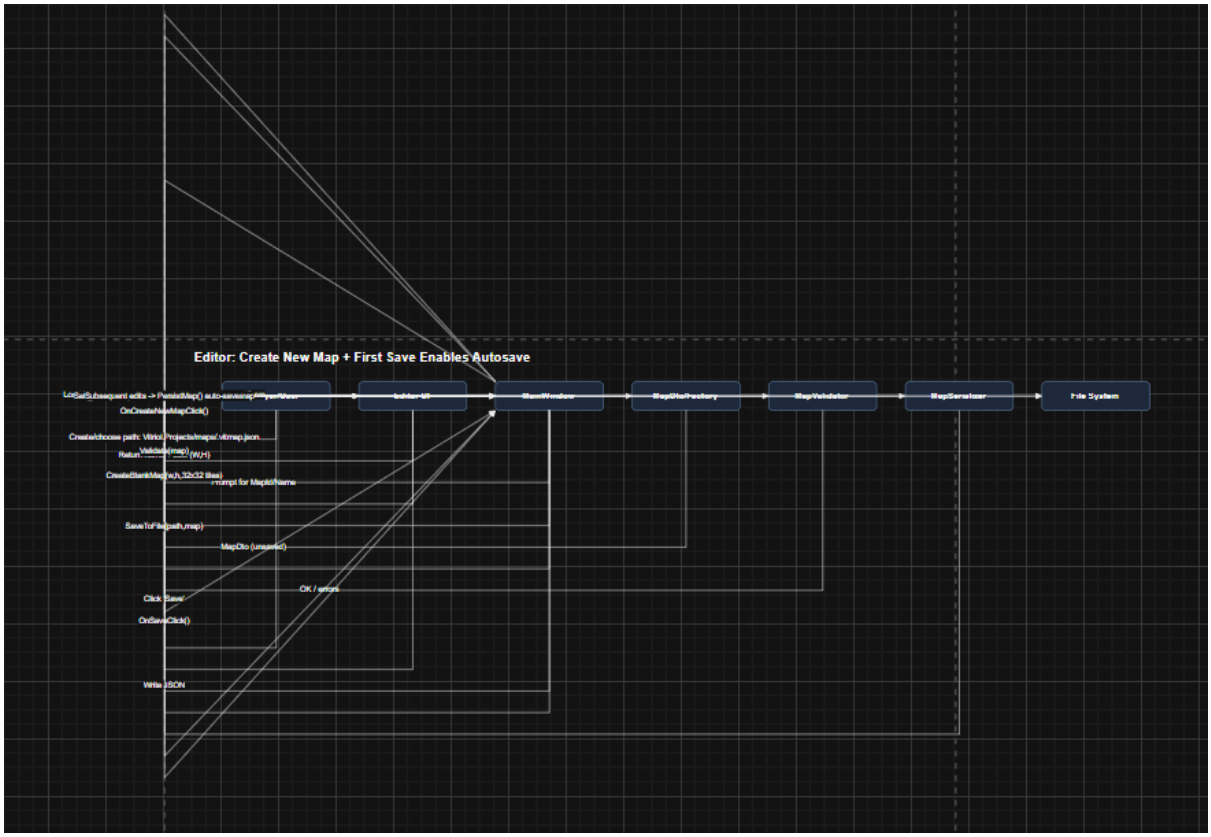
- Minimal HUD
- Explicit map loading
- Scrollable viewport with camera follow

The resulting interfaces prioritise clarity, task efficiency, and predictable behaviour, aligning directly with the usability objectives defined earlier in the design specification.

Behaviour and Control Flow: Sequence Diagrams

Map Creation: New Map vs Sample Map

Figure 19: Sequence – Map Creation and Initial Save State



This sequence diagram illustrates the lifecycle of a newly created map compared to the predefined sample map.

A key behavioural rule is enforced:

- Newly created maps must be manually saved at least once before autosave is enabled.
- The sample map always supports autosave.

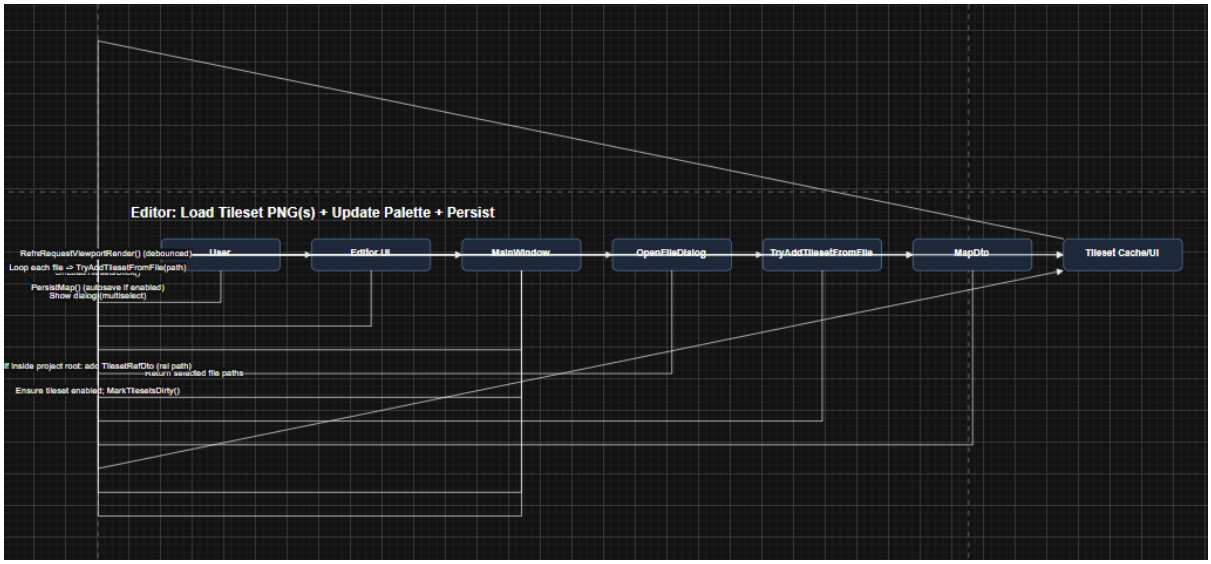
The primary risk during map creation is accidental persistence of invalid or incomplete data. The sequence therefore includes:

- Validation of map dimensions
- Construction of a safe, initialised tile grid
- Controlled activation of autosave only after a successful manual save

This ensures that file creation is deliberate and prevents unintended proliferation of incomplete map files.

Tileset Loading: Editor

Figure 20: Sequence – Tileset Selection, Validation, Referencing, Caching



This sequence diagram demonstrates the two-stage tileset loading model:

1. Reference registration (relative path storage)
2. Asset loading and caching

Tileset handling is intentionally separated into **reference** and **load** phases. This enables the editor to:

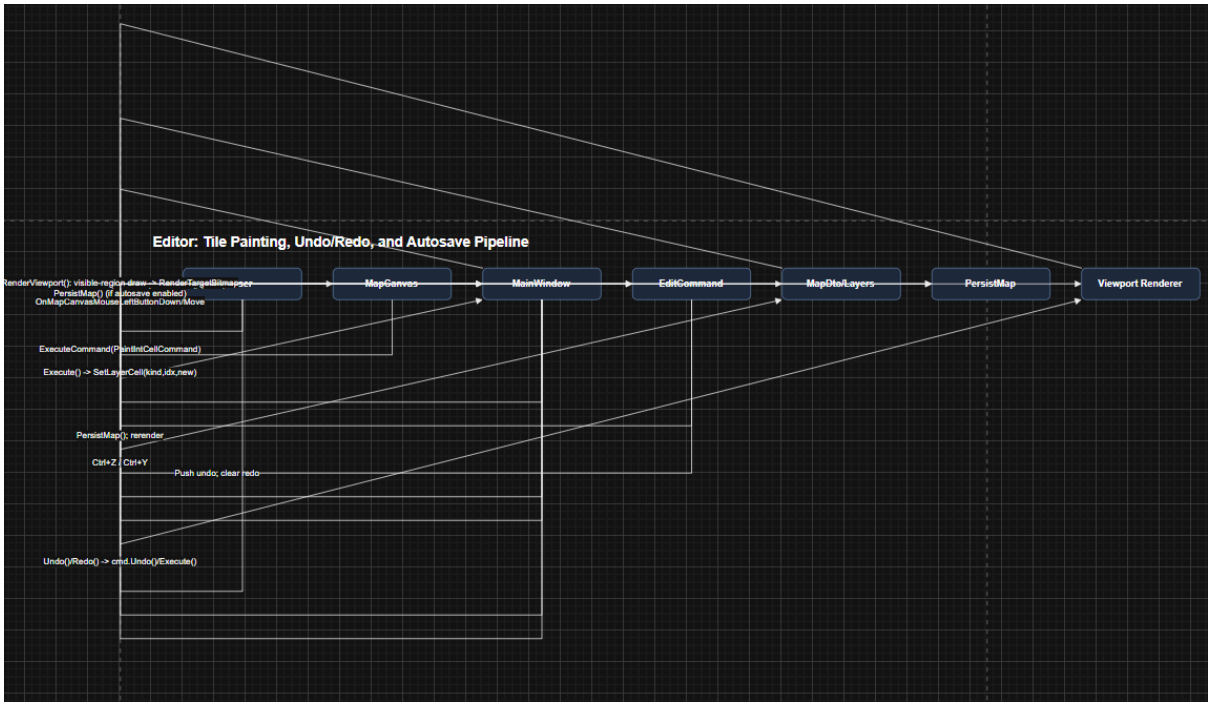
- Unload tilesets from memory to reduce resource usage
- Maintain stable global tile indices
- Prevent corruption when a tileset is toggled off

Stable indexing ensures that tile references stored within the map remain valid regardless of whether the tileset image is currently loaded into memory.

This design balances memory efficiency with structural consistency.

Tile Painting + Undo/Redo + Autosave

Figure 21: Sequence – Paint Action Routed Through Command + Persistence Rules



This sequence represents the convergence of multiple subsystems:

- Input handling
- Command stack
- Rendering updates
- Persistence rules

When a tile is painted:

1. Input is captured.
2. A command object is created.
3. The command mutates the data model.
4. The change is pushed onto the undo stack.
5. Rendering is refreshed.
6. Persistence rules determine whether autosave is triggered.

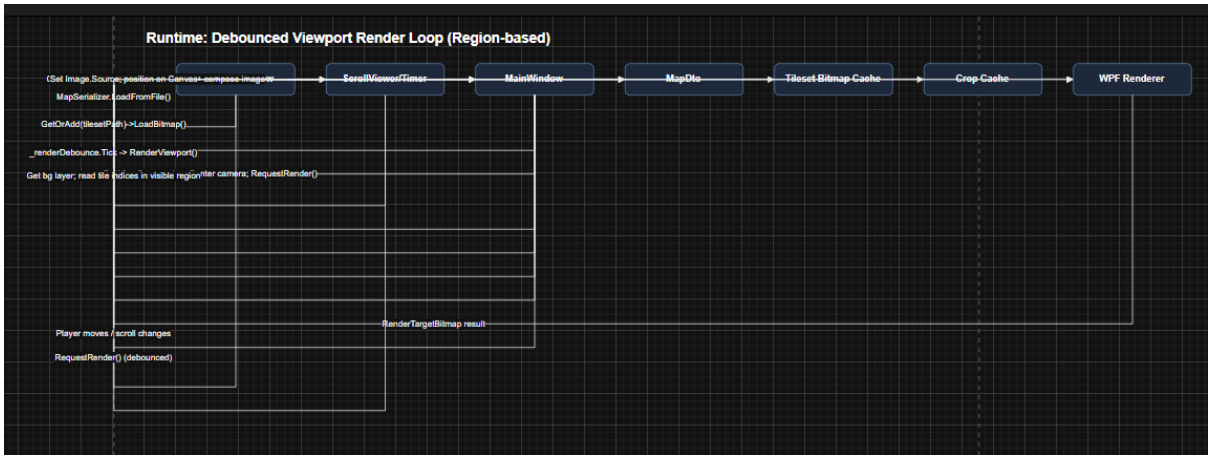
Explicitly modelling this flow ensured that:

- The data model is updated exactly once per tile change.
- Undo/Redo replays actions through the same execution pathway, avoiding special-case logic.
- Autosave behaviour adheres strictly to the defined rules for sample versus user-created maps.

This structured approach prevents divergence between editing actions and persistence behaviour.

Runtime Render Loop: Viewport-Only Rendering

Figure 22: Sequence – Runtime Viewport Rendering and Cache Usage



This diagram illustrates region-based viewport rendering.

The runtime performs:

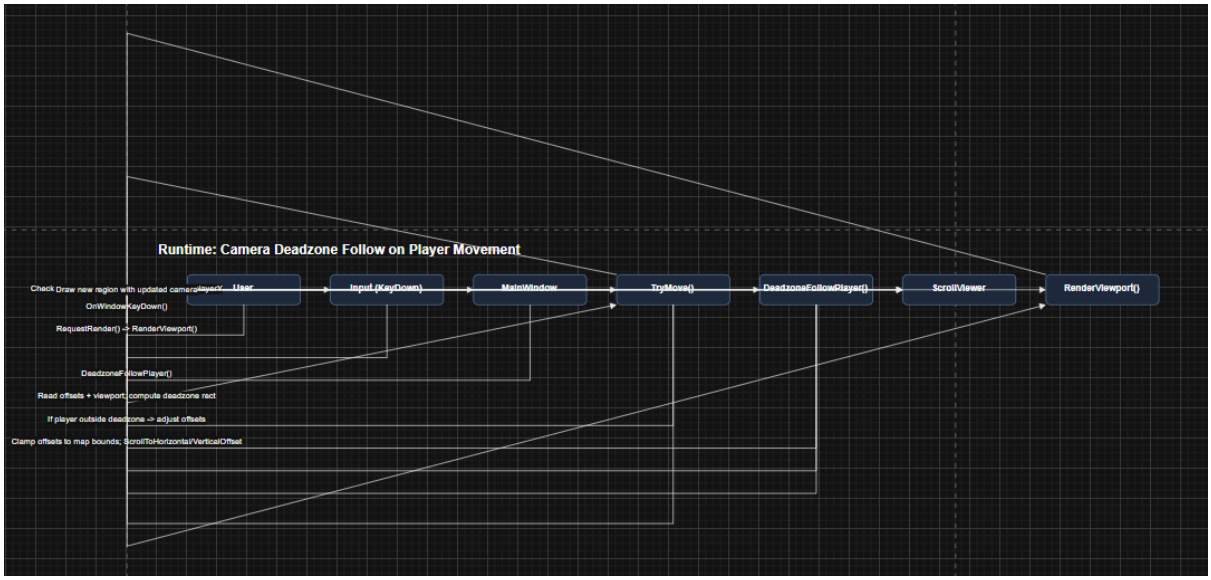
- Region calculation based on camera position
- Cropped tile selection
- Cached tile reuse where possible
- Single composited image output

Both the runtime and editor employ viewport-only rendering. This significantly improves performance for larger maps by limiting draw operations to visible tiles rather than the full grid.

Without region-based rendering, performance would degrade proportionally with map size, even when most tiles remain off-screen.

Camera Deadzone Follow

Figure 23: Sequence – Deadzone Follow Behaviour and Clamping



This diagram demonstrates the deadzone-based camera system.

The camera does not continuously track the player's exact position. Instead:

- A central deadzone region is defined within the viewport.
- The camera offset updates only when the player exits this region.
- Camera movement is clamped to map boundaries.

This prevents constant micro-adjustments and produces smoother perceived motion.

By shifting the camera only when necessary, the design reduces jitter and maintains visual stability, particularly during small player movements.

Design Justification Summary

Traceability: Design Decisions

The following architectural and behavioural decisions directly address the functional and non-functional requirements defined earlier. Each was selected deliberately to balance usability, maintainability, and performance.

- **Stable DTOs + Single Serialiser** -> Ensures both the editor and runtime interpret map files identically, eliminating format drift. By enforcing a single schema definition, long-term maintainability is improved and compatibility risks are reduced.
- **Flat Tile Arrays (1D Representation)** -> Chosen for faster indexing, simplified JSON serialisation, and predictable memory layout. This reduces overhead compared to nested 2D structures while maintaining conceptual clarity through coordinate conversion logic.
- **Command Pattern for Edits** -> Provides deterministic Undo/Redo behaviour across tile painting and event modification. This prevents inconsistent state

restoration and centralises mutation logic, improving reliability and debugging clarity.

- **Viewport-Only Rendering** -> Ensures large maps remain performant by limiting draw operations to visible regions. This avoids excessive WPF element creation and prevents frame-rate degradation proportional to map size.
- **Tileset Reference vs Load State Separation** -> Maintains stable tile indexing even when tilesets are toggled in memory. This prevents structural corruption of maps and supports efficient memory management.
- **Explicit Save Rules (Manual Save Before Autosave)** -> Prevents accidental file creation and unintended project folder pollution, while preserving rapid iteration through the always-autosaving sample map. This balances safety and workflow efficiency.
- **Runtime Manual Load Button** -> Introduced to allow explicit map selection at runtime, improving usability, decoupling runtime behaviour from editor state, and increasing testing flexibility.

Each of these decisions reinforces separation of concerns and predictable behaviour across the system.

Architectural Trade-offs and Alternatives Considered

Several alternative approaches were evaluated during the design phase:

- Embedding logic inside DTOs was rejected in favour of passive models to prevent cross-layer coupling.

- Using full 2D tile arrays was considered but rejected due to increased serialisation complexity and reduced indexing efficiency.

- Automatic autosave for all new maps was considered but rejected because it risked accidental file proliferation and loss of intentional naming control.

- A fully dynamic runtime auto-loading model was replaced with explicit loading to improve user agency and deployment realism.

These trade-offs demonstrate that the final design is the result of deliberate decision-making rather than incremental accumulation.

Known Constraints

While the system meets its defined objectives, several constraints are acknowledged:

- **Grid Rendering Precision** – Grid lines may visually degrade during zoom transitions due to pixel snapping behaviour within WPF's rendering pipeline.

- **Tileset File Dependency** – Tilesets referenced by maps must exist as PNG files within
  Vitriol.Projects/assets/tilesets/.
  Missing assets will result in validation failure or incomplete rendering.
- **Flexible Event Command Schema** – The command dictionary structure supports extensibility but requires strict validation to prevent malformed argument structures or inconsistent parameter usage.

These constraints are acceptable within the scope of the project and do not compromise core functionality, but they define operational boundaries for safe usage.